

Closure , Callback and Promises in JS

Closure:

- A closure is an inner function that has access to the outer (enclosing) function's variables
- The closure has three scopes, all part of the same chain:
 - o it has access to its own scope (variables defined between its curly brackets),
 - o it has access to the outer function's variables
 - o and it has access to the global variables
- The inner function has access not only to the outer function's variables, but also to the outer function's parameters

Example

```
function showName (firstName, lastName) {  
  
    var nameIntro = "Your name is ";  
  
    // this inner function has access to the outer function's variables, including the  
    // parameter  
  
    function makeFullName () {  
  
        return nameIntro + firstName + " " + lastName;  
  
    }  
  
    return makeFullName ();  
  
}  
  
showName ("Sachin", "Tendulkar"); // Your name is Sachin Tendulkar
```

Callback:

A callback function is a function which is:

- passed as an argument to another function
- is invoked after some kind of event
- once its parent function completes, the function passed as an argument is then called

Example

//the callback function

```
function done(){  
    console.log("Done");  
}
```

//the parent function

```
function increment(num, callBack){  
    for(var i = 0; i <= num; i++){  
        console.log(i);  
    }  
    return callBack();  
}
```

//the callback function is passed to the increment function

```
increment(10, done);
```

Important Point before promises:

- Normally code is *synchronous* - one statement executes and there is a guarantee that the next statement will execute immediately afterwards
- With *asynchronous* operations, you should assume that you have no idea when the operation will complete. You can't even assume that just because you send out one request first, and another request second, that they will return in that order
- Callbacks are the standard way of handling asynchronous code in JavaScript, but promises are the best way to handle asynchronous code.

Promises:

- A promise is an object which represents an eventual (future) value.
- A promise represents the result of an asynchronous operation. A promise is in one of three different states:
 - Pending (unfulfilled yet , still being computed)
 - fulfilled (resolved , succeeded)
 - rejected (an error happened)
- Once a promise is fulfilled or rejected, it is immutable (i.e. it can never change again)
- We use “ new Promise” to construct the promise, the constructor is called immediately with two arguments –
 - one that resolve(fulfills) the promise
 - and the other that rejects the promise

Example:

```
var p = new Promise(function(resolve, reject) {  
    // Do an async task async task and then...  
    if(/* good condition */) {  
        resolve('Success!');  
    }  
    else {  
        reject('Failure!');  
    }  
});  
p.then(function() {  
    /* do something with the result */  
}).catch(function() {  
    /* error :( */  
});
```

Example Try this

```
var p = new Promise(function(resolve, reject) {  
    // A mock async action using setTimeout  
    setTimeout(function() { resolve(10); }, 3000);  
});  
p.then(function(result) {  
    console.log(result);  
});
```